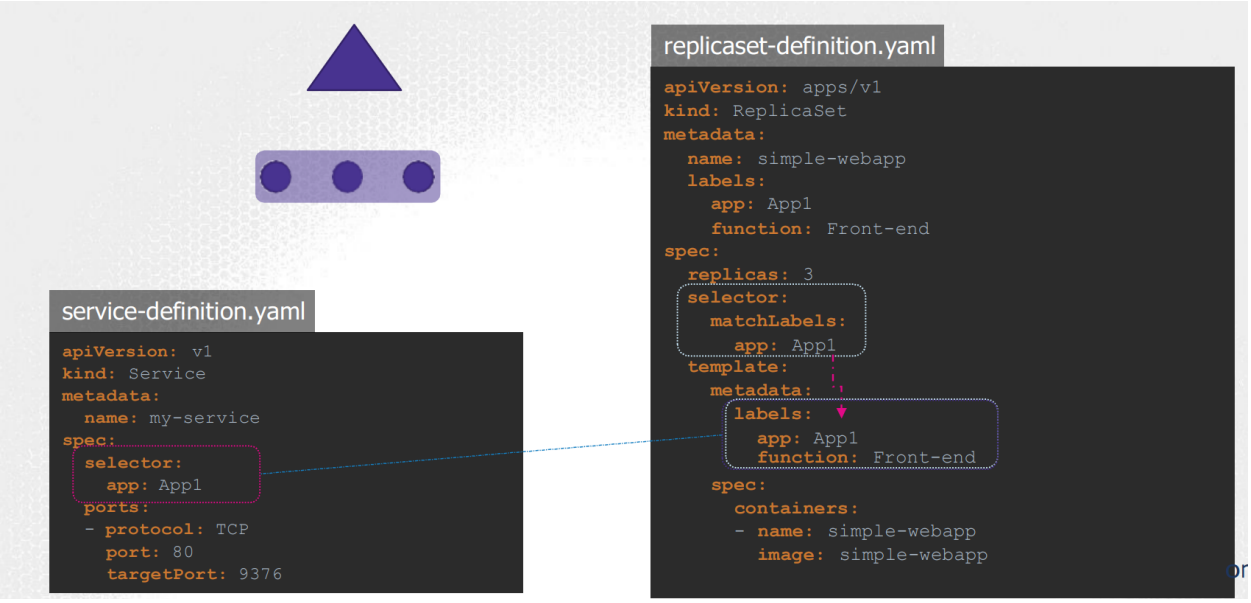
KUBERNETES CKAD 2

1. POD DESIGN



replicaset-definition.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: simple-webapp
  labels:
    app: App1
    function: Front-end
  annotations:
      buildversion: 1.34
spec:
  replicas: 3
  selector:
    matchLabels:
      app: App1
  template:
    metadata:
      labels:
        app: App1
        function: Front-end
    spec:
      containers:
      - name: simple-webapp
        image: simple-webapp
```

job-definition.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: random-error-job
spec:
  completions: 3
  parallelism: 3
  template:
    spec:
      containers:
        - name: random-error
          image: kodekloud/random-error

      restartPolicy: Never
```

cron-job-definition.yaml

```yaml
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: reporting-cron-job
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      completions: 3
      parallelism: 3
      template:
        spec:
          containers:
            - name: reporting-tool
              image: reporting-tool

          restartPolicy: Never
```

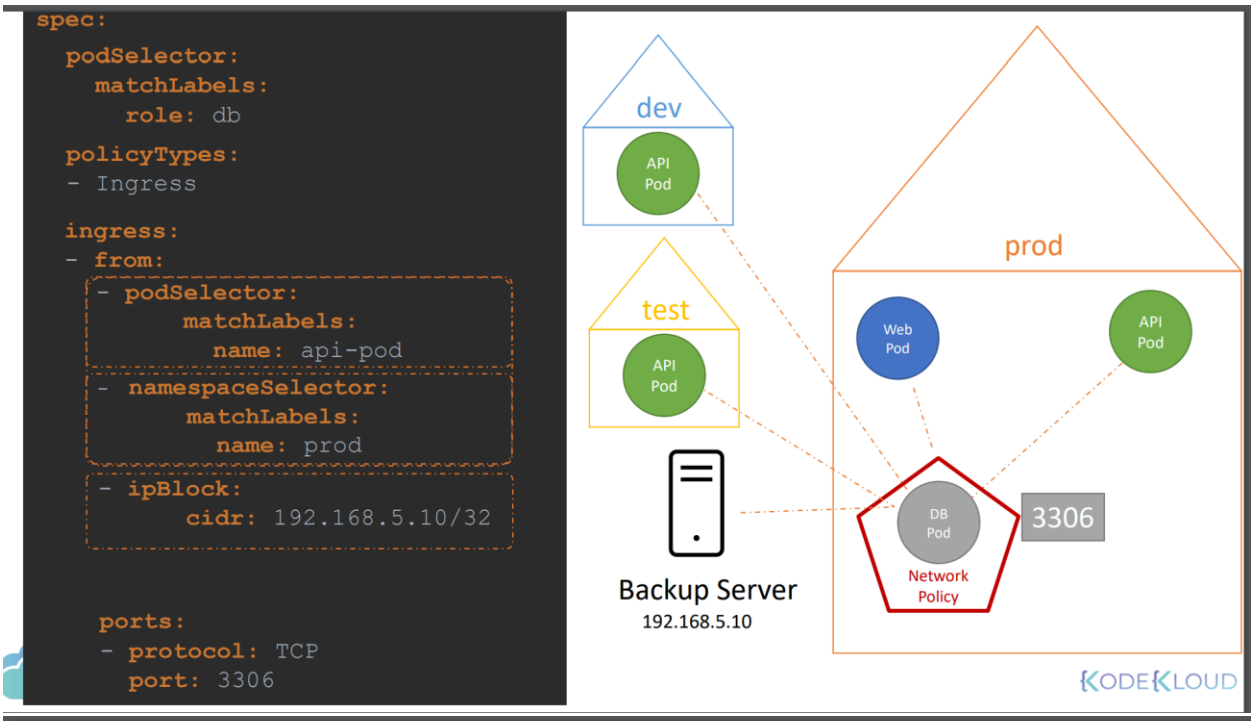

kubectl run nginx1 --image=nginx --restart=Never -l app=v1 Create a pod named nginx1 with the label app=v1.
kubectl get po --show-labels Show all pods with their labels.
kubectl label po nginx2 app=v2 --overwrite Change the label of the pod nginx2 to app=v2.
kubectl get po -l app=v2 Get all pods with the label app=v2.
kubectl label po -l "app in (v1,v2)" tier=web Add the label tier=web to pods with labels app=v1 or app=v2.
kubectl annotate po nginx2 owner=marketing Add the annotation owner=marketing to the pod nginx2.
kubectl label po -l app app- Remove the app label from all pods with the app label.
kubectl annotate po nginx1 description='my description' Add an annotation description='my description' to

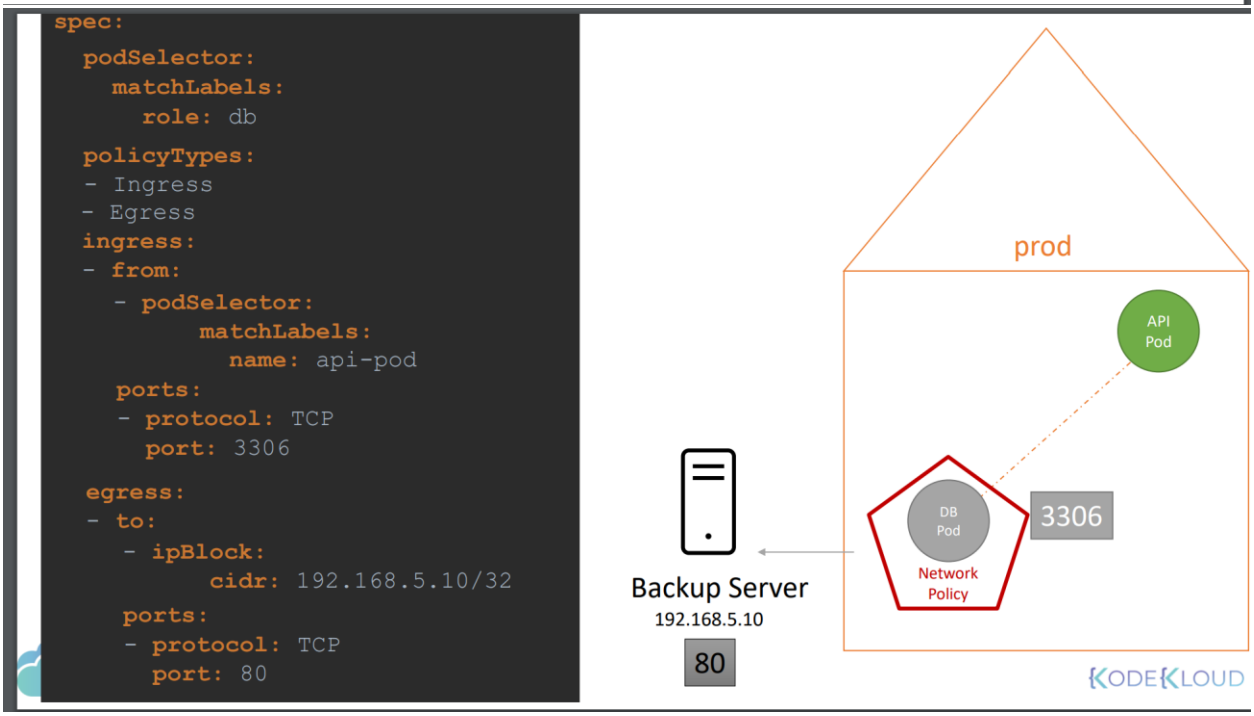
the pod nginx1.
kubectl annotate po nginx1 --list List all annotations on the pod nginx1.
kubectl annotate po nginx{1..3} description- owner- Remove the annotations description and owner from pods nginx1, nginx2, and nginx3.
kubectl delete po nginx{1..3} Delete pods nginx1, nginx2, and nginx3.
kubectl label nodes <node-name> accelerator=nvidia-tesla-p100 Add the label accelerator=nvidia-tesla-p100 to a node.
kubectl get nodes --show-labels Show all nodes with their labels.
kubectl taint node node1 tier=frontend:NoSchedule Add a taint tier=frontend:NoSchedule to the node node1.
kubectl taint node node1 tier=frontend:NoSchedule- Remove the taint tier=frontend:NoSchedule from the node node1.
kubectl create deployment nginx --image=nginx:1.18.0 --replicas=2 --port=80 Create a deployment named nginx with 2 replicas, using image nginx:1.18.0, exposing port 80.
kubectl get deploy nginx -o yaml View the YAML definition of the nginx deployment.
kubectl set image deploy nginx nginx=nginx:1.19.8 Update the deployment nginx to use the image nginx:1.19.8.
kubectl rollout status deploy nginx Check the rollout status of the deployment nginx.
kubectl rollout undo deploy nginx Undo the latest rollout for the deployment nginx.
kubectl scale deploy nginx --replicas=5 Scale the deployment nginx to 5 replicas.
kubectl autoscale deploy nginx --min=5 --max=10 --cpu-percent=80 Autoscale the deployment nginx between 5 and 10 replicas, targeting 80% CPU usage.
kubectl rollout pause deploy nginx Pause the rollout of the deployment nginx.
kubectl rollout resume deploy nginx Resume the rollout of the deployment nginx.
kubectl create job pi --image=perl:5.34 -- perl -Mbignum=bpi -wle 'print bpi(2000)' Create a job named pi to calculate the value of π.
kubectl get jobs View all jobs in the cluster.
kubectl logs job/pi View the logs of the job pi.
kubectl delete job pi Delete the job pi.
kubectl create cronjob busybox --image=busybox --schedule="*/1 * * * *" -- /bin/sh -c 'date; echo Hello from the Kubernetes cluster' Create a cron job named busybox that runs every minute and prints a message.
kubectl get cj List all cron jobs in the cluster.
kubectl delete cj busybox Delete the cron job busybox.
kubectl create job --from=cronjob/busybox sample-job Create a job named sample-job from the cron job busybox.

2. Services

```yaml
apiVersion: v1
kind: Service
metadata:
  name: clusterip-service
spec:
  selector:
    app: myapp
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
  type: ClusterIP
---
apiVersion: v1
kind: Service
metadata:
  name: nodeport-service
spec:
  selector:
    app: myapp
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
    nodePort: 30080
  type: NodePort
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: loadbalancer-service
spec:
  selector:
    app: myapp
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
  type: LoadBalancer
---
```

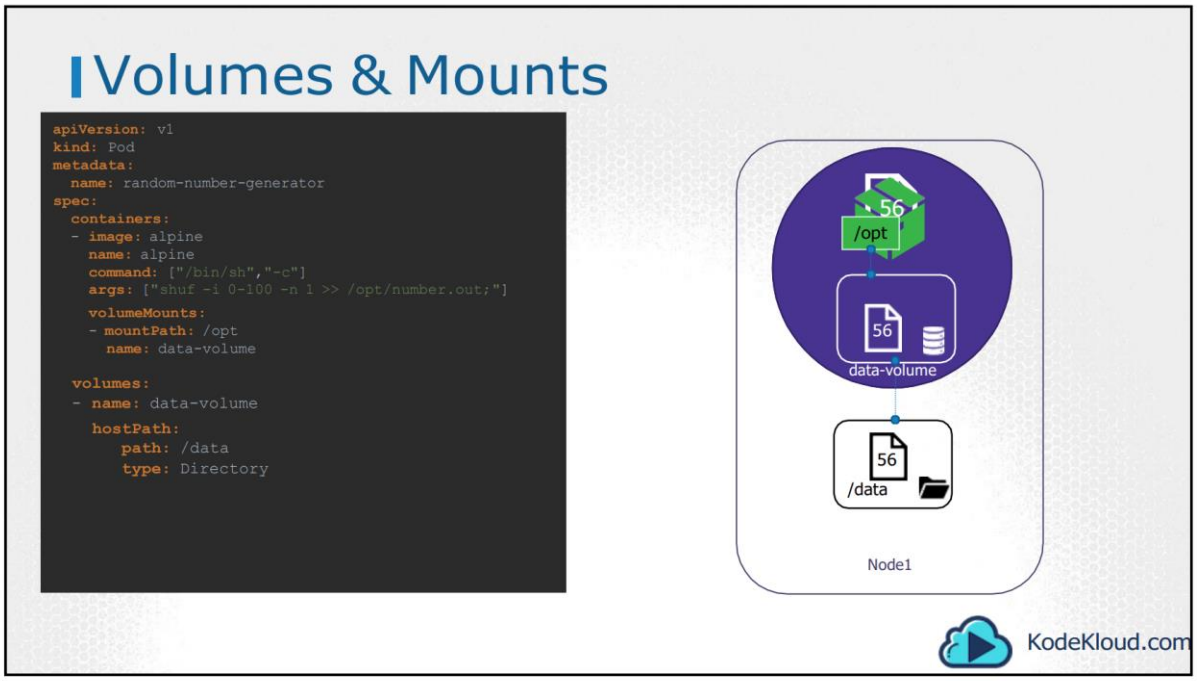




kubectl run nginx --image=nginx --restart=Never --port=80 --expose Create a pod named nginx with image nginx and expose it as a ClusterIP service on port 80.
kubectl get svc nginx Display details about the service created for the nginx pod.
kubectl get ep Check the endpoints associated with all services in the cluster.
IP=$(kubectl get svc nginx --template={{.spec.clusterIP}}) Get the ClusterIP of the nginx service and store it in a variable.
kubectl run busybox --rm --image=busybox -it --restart=Never --env="IP=$IP" -- wget -O- $IP:80 --

timeout 2 Create a temporary busybox pod and use wget to hit the nginx service on its ClusterIP.
kubectl patch svc nginx -p '{"spec":{"type":"NodePort"}}' Change the type of the nginx service from ClusterIP to NodePort.
wget -O- <NODE_IP>:<NODE_PORT> Access the nginx service using the Node's IP and NodePort.
kubectl delete svc nginx Delete the nginx service.
kubectl delete pod nginx Delete the nginx pod.
kubectl create deploy foo --image=dgkanatsios/simpleapp --port=8080 --replicas=3 Create a deployment named foo with 3 replicas, using the image dgkanatsios/simpleapp, and expose port 8080 on its containers.
kubectl get pods -l app=foo -o wide List all pods with the label app=foo and include their IP addresses.
kubectl run busybox --image=busybox --restart=Never -it --rm -- sh Create a temporary interactive busybox pod to run commands.
wget -O- <POD_IP>:8080 Use wget within the busybox pod to connect to one of the foo deployment pods on port 8080.
kubectl expose deploy foo --port=6262 --target-port=8080 Expose the foo deployment as a ClusterIP service on port 6262, mapping to the target port 8080 of the containers.
kubectl get svc foo Display details of the foo service.
kubectl get endpoints foo Check the endpoints for the foo service to verify they match the pod IPs.
kubectl run busybox --image=busybox -it --rm --restart=Never -- sh Create a temporary busybox pod to test the foo service.
wget -O- foo:6262 Test the foo service using DNS within the busybox pod.
wget -O- <SERVICE_CLUSTER_IP>:6262 Test the foo service using its ClusterIP.
kubectl delete svc foo Delete the foo service.
kubectl delete deploy foo Delete the foo deployment.
kubectl create deployment nginx --image=nginx --replicas=2 Create an nginx deployment with 2 replicas.
kubectl expose deployment nginx --port=80 Expose the nginx deployment as a ClusterIP service on port 80.
kubectl create -f policy.yaml Apply a NetworkPolicy from the policy.yaml file to restrict access to the nginx pods.
kubectl run busybox --image=busybox --rm -it --restart=Never -- wget -O- http://nginx:80 --timeout 2 Create a temporary busybox pod to test access to the nginx service without the required labels.
kubectl run busybox --image=busybox --rm -it --restart=Never --labels=access=granted -- wget -O- http://nginx:80 --timeout 2 Create a temporary busybox pod with the label access=granted and test access to the nginx service.

3. Volumes



Create a busybox pod with command 'sleep 3600', save it on pod.yaml. Mount the PersistentVolumeClaim to '/etc/foo'. Connect to the 'busybox' pod, and copy the '/etc/passwd' file to '/etc/foo/passwd'

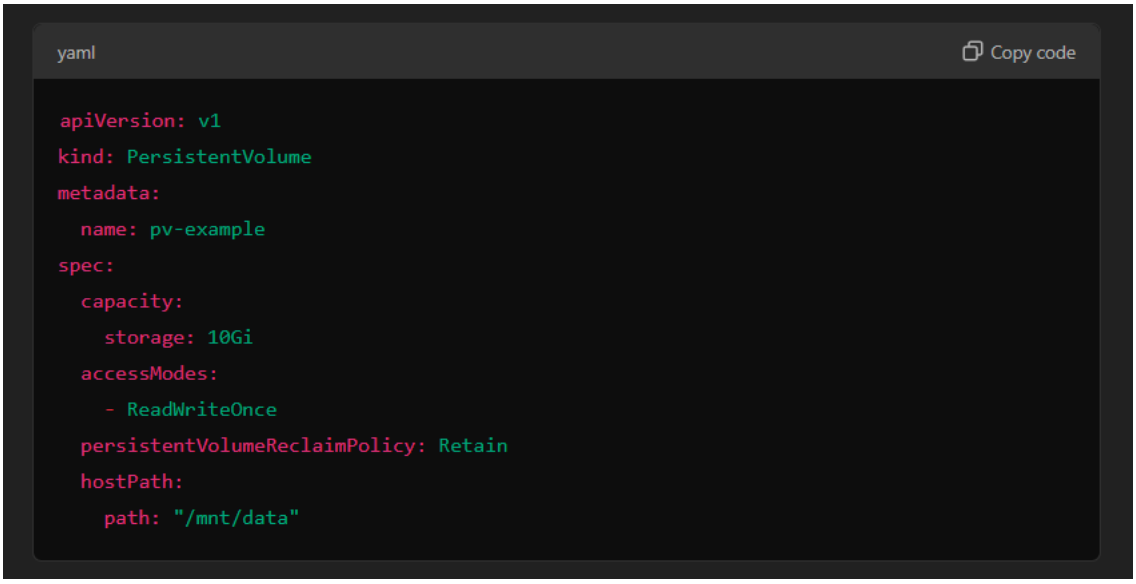
▼ show

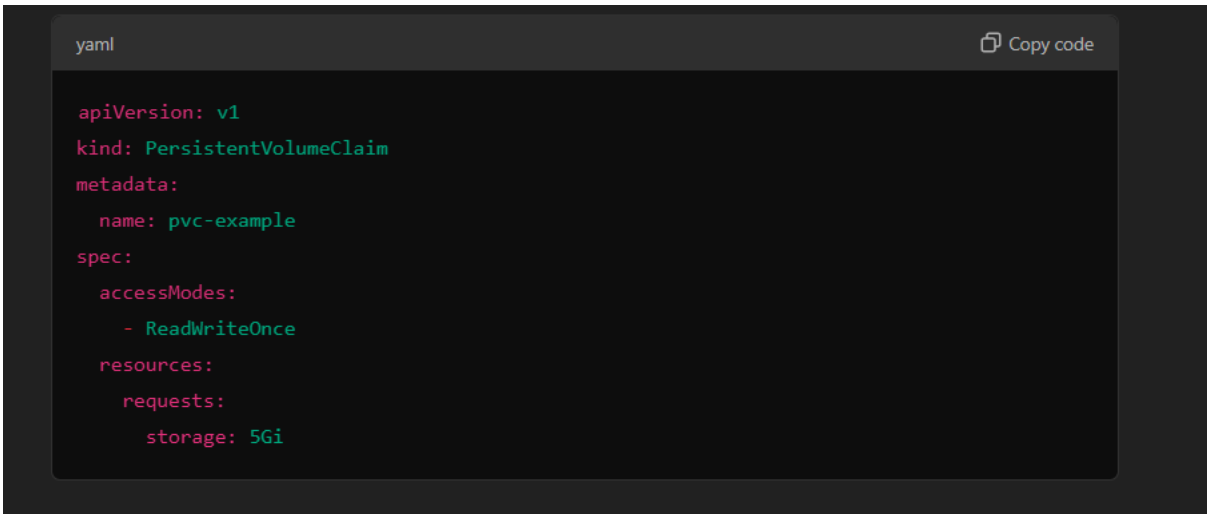
Create a skeleton pod:

```
kubectl run busybox --image=busybox --restart=Never -o yaml --dry-run=client -- /bin/sh -c 'sleep 3600' > pod.yaml
vi pod.yaml
```

Add the lines that finish with a comment:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: busybox
  name: busybox
spec:
  containers:
  - args:
    - /bin/sh
    - -c
    - sleep 3600
    image: busybox
    imagePullPolicy: IfNotPresent
    name: busybox
    resources: {}
    volumeMounts: #
    - name: myvolume #
      mountPath: /etc/foo #
  dnsPolicy: ClusterFirst
  restartPolicy: Never
  volumes: #
  - name: myvolume #
    persistentVolumeClaim: #
      claimName: mypvc #
status: {}
```





4. HELM

helm create chart-test Create a basic Helm chart named chart-test.
helm install -f myvalues.yaml myredis ./redis Install a Helm chart located in the redis directory with values from myvalues.yaml. Name the release myredis.
helm list --pending -A List all pending Helm deployments across all namespaces.
helm uninstall -n namespace release_name Uninstall a Helm release in the specified namespace.
helm upgrade -f myvalues.yaml -f override.yaml redis ./redis Upgrade the redis release using the Helm chart in the redis directory with values from myvalues.yaml and override.yaml.
helm repo add [NAME] [URL] Add a Helm chart repository with the given name and URL.
helm repo list List all added Helm repositories.
helm repo remove [REPO1] Remove a specified Helm repository.

helm repo update Update all Helm repositories to fetch the latest charts.
helm repo index [DIR] Generate an index file for the charts in the specified directory.
helm pull [chart URL | repo/chartname] Download a Helm chart from the specified repository or URL.
helm pull --untar [repo/chartname] Download and extract a Helm chart from the specified repository.
helm repo add bitnami https://charts.bitnami.com/bitnami Add the Bitnami Helm chart repository to your list of Helm repositories.
helm show values bitnami/node Display the contents of the values.yaml file for the bitnami/node chart.
helm install mynode bitnami/node --set replicaCount=5 Install the bitnami/node chart with a release name mynode and set the number of replicas to 5 using the --set flag.

5. CRD

kubectl apply -f operator-crd.yml
This command applies the `operator-crd.yml` manifest to create a CustomResourceDefinition (CRD) named `operators.stable.example.com` in the Kubernetes API. The CRD defines a new custom resource called `Operator` with a schema (`email`, `name`, and `age`), scope (`Namespaced`), and names (`plural: operators`, `singular: operator`, `shortNames: op`).

kubectl apply -f operator.yml
This command applies the `operator.yml` manifest to create a custom resource of type `Operator`. The custom resource has the name `operator-sample` and a spec with the fields `email`, `name`, and `age`. This custom object conforms to the CRD schema defined in the previous step.

kubectl get operators
This command lists all custom resources of type `Operator` using the plural name `operators`. It provides an overview of all resources defined under the CRD.

kubectl get operator
This command lists all custom resources of type `Operator` using the singular name `operator`. It is an alternative way to fetch the same resources listed with the plural form.

kubectl get op
This command lists all custom resources of type `Operator` using the short name `op`. It is a convenient shorthand defined in the CRD for quickly accessing the resources.

- *Name* : `operators.stable.example.com`
- *Group* : `stable.example.com`
- *Schema*: `<email: string><name: string><age: integer>`
- *Scope*: `Namespaced`
- *Names*: `<plural: operators><singular: operator><shortNames: op>`
- *Kind*: `Operator`

▼ show

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  # name must match the spec fields below, and be in the form: <plural>.<group>
  name: operators.stable.example.com
spec:
  group: stable.example.com
  versions:
    - name: v1
      served: true
      # One and only one version must be marked as the storage version.
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                email:
                  type: string
                name:
                  type: string
                age:
                  type: integer
  scope: Namespaced
  names:
    plural: operators
    singular: operator
    # kind is normally the CamelCased singular type. Your resource manifests use this.
    kind: Operator
    shortNames:
    - op
```

Kubectl config

`kubectl config view`

```
apiVersion: v1

kind: Config
current-context: my-kube-admin@my-kube-playground


clusters:
- name: my-kube-playground
- name: development
- name: production


contexts:
- name: my-kube-admin@my-kube-playground
- Name: prod-user@production


users:
- name: my-kube-admin
- name: prod-user
```

`kubectl config use-context prod-user@production`

```
apiVersion: v1

kind: Config
current-context: prod-user@production


clusters:
- name: my-kube-playground
- name: development
- name: production


contexts:
- name: my-kube-admin@my-kube-playground
- Name: prod-user@production


users:
- name: my-kube-admin
- name: prod-user
```